ISSN 0032-9460, Problems of Information Transmission, 2011, Vol. 47, No. 1, pp. 28–33. © Pleiades Publishing, Inc., 2011.
Original Russian Text © M.A. Babenko, T.A. Starikovskaya, 2011, published in Problemy Peredachi Informatsii, 2011, Vol. 47, No. 1, pp. 33–39.

LARGE SYSTEMS

# Computing the Longest Common Substring with One Mismatch[1]

**M. A. Babenko and T. A. Starikovskaya**

*Chair of Mathematical Logic and Theory of Algorithms, Faculty of Mechanics and Mathematics, Lomonosov Moscow State University*
maxim.babenko@gmail.com tat.starikovskaya@gmail.com

Received May 7, 2010; in final form, August 24, 2010

**Abstract**—The paper describes an algorithm for computing longest common substrings of two strings $\alpha_1$ and $\alpha_2$ with one mismatch in $O(|\alpha_1||\alpha_2|)$ time and $O(|\alpha_1|)$ additional space. The algorithm always scans symbols of $\alpha_2$ *sequentially*, *starting from the first symbol*. The RAM model of computation is used.

**DOI:** 10.1134/S0032946011010030

## 1. INTRODUCTION

Computing the longest common substring is one of the basic problems in stringology. However, for applications such as bioinformatics and text analysis, the problem of computing longest common substrings with mismatches (insertions, deletions, and substitutions of symbols) is of much greater importance. There are several approaches to this problem, for instance, dynamic programming. A solution based on dynamic programming computes longest common approximate substrings using time and space proportional to the product of strings' lengths [1].

The algorithm described in the paper has the same running time and computes the longest common substrings of two strings with one mismatch. Additional space used by the algorithm is proportional to the least of strings' lengths.

The most general form of the problem in question is as follows.

Given two strings $\alpha_1$ and $\alpha_2$, compute substrings $\gamma_1$ and $\gamma_2$ of $\alpha_1$ and $\alpha_2$, respectively, that differ in at most one symbol and have maximum lengths.

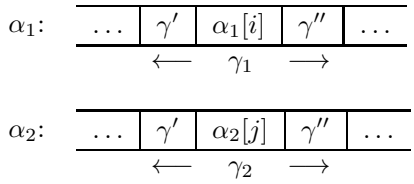
Let us assume that the length of $\alpha_2$ is sufficiently larger than that of $\alpha_1$. The considered algorithm makes several passes over $\alpha_2$, but each time reads the string *sequentially from the left to the right*, *starting from the first symbol*. This condition is essential for applications that store $\alpha_2$ in external memory, since random access becomes time-expensive in this case.

Hereafter, $|\alpha_1|$ is denoted by $n_1$ and $|\alpha_2|$ is denoted by $n_2$ ($n_2 \geq n_1$). The running time of the presented algorithm is $O(n_1 n_2)$, and the additional memory used by the algorithm is $O(|\alpha_1|)$.

**Preliminaries.** We assume that a finite nonempty set (*alphabet*) $\Sigma$ is fixed. Elements of this set are called *letters* or *symbols*. A finite ordered sequence of letters (possibly empty) is called a *string*.

We use Greek letters to denote strings. Letters in a string are numbered starting from 1; for example, letters of $\alpha$ (which is of length $k$) are denoted by $\alpha[1], \ldots, \alpha[k]$. The length $k$ of $\alpha$ is denoted by $|\alpha|$. The *substring* of $\alpha$ from position $i$ to position $j$ (inclusive) is denoted by $\alpha[i : j]$.

[1] Supported in part by the Russian Foundation for Basic Research, project no. 09-01-00709-a.




Strings $\alpha_1$ and $\alpha_2$ and their substrings $\gamma_1$ and $\gamma_2$

**Definition 1.** A string $\beta = \alpha[1 : j]$ is called a *prefix* of $\alpha$ and is denoted by $\alpha[ : j]$. Similarly, a string $\beta = \alpha[i : |\alpha|]$ is called a *suffix* of $\alpha$ and is denoted by $\alpha[i : ]$.

The length of the longest common prefix of strings $\alpha$ and $\beta$ is denoted by $lcp(\alpha, \beta)$, and the length of the longest common suffix of strings $\alpha$ and $\beta$ is denoted by $lcs(\alpha, \beta)$.

We assume that a linear order on $\Sigma$ is fixed. This *lexicographic* order on $\Sigma$ can be extended in a standard way to the set of strings in $\Sigma$. We write $\alpha < \beta$ to denote that $\alpha$ lexicographically precedes $\beta$; similarly for other relation signs.

We assume the usual RAM model of computation [2]. Also, symbols in $\Sigma$ are treated just as integers in range $\{1, \ldots, |\Sigma|\}$, so one can compare any pair of them in $O(1)$ time.

First we give a naive algorithm for computing longest common substrings with one mismatch. The algorithm simply compares all substrings of length $d$ symbol by symbol, $1 \le d \le n_1$. Namely, each substring of length $d$ of $\alpha_1$ is compared with each substring of length $d$ of $\alpha_2$. If there exists a substring of $\alpha_1$ and a substring of $\alpha_2$, both of length $d$, differing in at most one symbol, then the length of $\gamma_1$ and $\gamma_2$ is at least $d$. We run this procedure for all $d$ from 1 to $\alpha_1$ to obtain the desired result.

The naive algorithm can be easily adjusted to read $\alpha_2$ sequentially, starting from the first symbol. Namely, we fix a substring of length $d$ of $\alpha_1$ and compare it with the strings $\alpha_2[1 : d]$, $\alpha_2[2 : d+1]$, $\ldots$, $\alpha_2[n_2 - d + 1 : n_2]$. Knowing symbols of $\alpha_2[i : i + d - 1]$, it suffices to read $\alpha_2[i + d]$ to get the substring $\alpha_2[i+1 : i+d]$. Therefore, we read the first $d$ symbols of $\alpha_2$ to get the substring $\alpha_2[1 : d]$ and then read one symbol per step.

The running time of the naive algorithm is $O(n_1^2 n_2^2)$, the amount of additional memory used is $O(1)$. Our goal is to speed up the algorithm by a factor of $n_1 n_2$.

## 2. ALGORITHM

The idea of the algorithm is as follows. Assume that strings $\gamma_1$ and $\gamma_2$ have coinciding parts $\gamma'$ and $\gamma''$ (and there is only one position in which $\gamma_1$ and $\gamma_2$ might differ). Let the position of the mismatch in $\alpha_1$ be $i$ and in $\alpha_2$ be $j$. Obviously, $\gamma'$ is the longest common suffix of $\alpha_1[ : i - 1]$ and $\alpha_2[ : j - 1]$, and $\gamma''$ is the longest common prefix of $\alpha_1[i + 1 : ]$ and $\alpha_2[j + 1 : ]$ (see the figure).

In fact, we do not check if the symbols $\alpha_1[i]$ and $\alpha_2[j]$ are the same. We use the position of a mismatch only as a label dividing $\gamma_1$ and $\gamma_2$ into two parts. Therefore, the algorithm computes $\gamma_1$ and $\gamma_2$ that differ in zero or one symbol.

We denote $lcs(\alpha_1[ : k], \alpha_2[ : l])$ by $lcs(k, l)$ and $lcp(\alpha_1[k : ], \alpha_2[l : ])$ by $lcp(k, l)$. Assume that for positions $i$ and $j$ in $\alpha_1$ and $\alpha_2$, respectively, we know $lcs(i - 1, j - 1)$ and $lcp(i + 1, j + 1)$. Then $s(i, j) = lcs(i - 1, j - 1) + lcp(i + 1, j + 1) + 1$ is the length of the longest common substring of $\alpha_1$ and $\alpha_2$ with a mismatch in positions $i$ and $j$.

Denote the length of the longest common substring with a mismatch in position $i$ of string $\alpha_1$ by $d_i$. Obviously, $d_i = \max\limits_{1 \le j \le n_2} s(i, j)$. The value of $d_i$ can be computed in $O(n_2)$ steps, sequentially running the LCP algorithm (which computes $lcp(i + 1, j + 1)$) and the LCS algorithm (which computes $lcs(i - 1, j - 1)$). The desired length of the longest common substring is $\max\limits_{1 \le i \le n_1} d_i$.

Keeping positions of mismatches in $\alpha_1$ and $\alpha_2$ that give the maximum of $d_i$ allows one to compute the longest common substring itself.

**Algorithm.** Computing the longest common substring with one mismatch

```
MaxLength ← 0
for all i ← 1 ... n_1 do
    Build a suffix tree for α_1[1 : i − 1]
    Compute Z-values for α_1[i + 1 : n_1]
    Compute Z-values for α_1[1 : i − 1]^{−1}
    for all j ← 1 ... n_2 do
        Compute lcp(i + 1, j + 1)
        Compute lcs(i − 1, j − 1)
        s(i, j) ← 1 + lcp(i + 1, j + 1) + lcs(i − 1, j − 1)
        if MaxLength < s(i, j) then
            MaxLength ← s(i, j)
            Imax ← i
            Jmax ← j
        end if
    end for
end for
```

We also give a pseudocode for the algorithm. Here *MaxLength* stands for the length of longest common substrings with one mismatch, and *Imax* and *Jmax* stand for positions of strings $\alpha_1$ and $\alpha_2$ giving the maximum of the length. The algorithm uses the notions of suffix trees and $Z$-values, which will be defined later.

We describe the subroutines for computing $lcp(i+1, j+1)$ and $lcs(i-1, j-1)$ separately. Still in the main algorithm they are run in parallel, which is made possible by the structure of these subroutines. The LCP algorithm computes $lcp(i+1, j+1)$ for a fixed $i$ and all $j = 1, \ldots, n_2$ and is described in Section 3. The LCS algorithm computes $lcs(i-1, j-1)$ for a fixed $i$ and all $j = 1, \ldots, n_2$ and is described in Section 4.

## 3. LCP ALGORITHM

In this section we give a high-level description of the LCP algorithm, which computes $lcp(i+1, j+1)$ for a fixed $i$ and all $j = 1, \ldots, n_2$. Its linear time bound is proved in [1].

**Definition 2.** For a given string $\beta$ and for any $j$, $1 < j \le |\beta|$, define $Z_j(\beta)$ to be equal to $lcp(\beta, \beta[j : ])$.

If $Z_j > 0$, we define the *Z-box* starting at position $j$ as $\alpha[j : j + Z_j(\beta) - 1]$.

We denote the rightmost endpoint of $Z$-boxes that begin at or before position $j$ by $r_j$. Let $l_j$ stand for the position of the left end of the $Z$-box that ends at $r_j$.

$Z$-values are computed sequentially for $k = 1, 2, \ldots, |\beta|$. In each step, the algorithm computes $Z_{k+1}(\beta)$, $r_{k+1}$, and $l_{k+1}$ using the values $Z_k(\beta)$, $r_k$, and $l_k$. Note that, by the definition of $Z$-values, the strings $\beta[k+1 : r_k]$ and $\beta[k - l_k : r_k - l_k - 1]$ coincide. Therefore, the first $\min(Z_{k-l_k}(\beta), r_k - l_k - 1)$ symbols of the strings $\beta$ and $\beta[k+1 : ]$ are the same. To compute $Z_{k+1}(\beta)$, the algorithm sequentially compares pairs of symbols of $\beta$ and $\beta[k+1 : ]$ starting at position $\min(Z_{k-l_k}(\beta), r_k - l_k - 1)$ until it encounters the first mismatch. The last position before the mismatch is $Z_{k+1}(\beta)$. If $r_k$ is less than $k + Z_{k+1}(\beta)$, then $r_{k+1} = k + Z_{k+1}(\beta)$ and $l_{k+1} = k + 1$; otherwise, $r_{k+1} = r_k$ and $l_{k+1} = l_k$.

To compute $lcp(i+1, j)$ for $j = 1, \ldots, n_2$, we consider the string $\beta = \alpha_1[i+1 : ]\$\alpha_2$, where $\$ \notin \Sigma$. By definition, $Z_{n_1+j-i}(\beta)$ is equal to $lcp(i+1, j)$. For the considered string $\beta$, keeping only

the first $n_1 - i$ of its $Z$-values is sufficient to compute any other $Z$-value. Indeed, for any $k > n_1 - i$, the value $k - l_k$ is not greater than $n_1 - i$ (because of \$). Therefore, to compute $Z_{k+1}(\beta)$, the algorithm needs only $Z_k(\beta)$, $r_k$, $l_k$, and $Z_j(\beta)$, $j \le n_1 - i$. Note that it is not necessary to store $\beta$ in memory.

Hence, $Z$-values for $\beta$ can be computed in time $O(|\beta|) = O(n_2)$. As is mentioned above, only $O(n_1 - i) = O(n_1)$ additional memory is used.

## 4. LCS ALGORITHM

In this section we describe the LCS algorithm, which computes $lcs(i-1, j-1)$ for a fixed $i$ and all $j = 1, \ldots, n_2$. Note that it is impossible to use the LCP algorithm for reversed strings $\alpha_1$ and $\alpha_2$, since we want to read $\alpha_2$ only from left to right.

**Definition 3.** A *pre-occurrence* of a string $P$ at position $j$ in a string $\beta$ is an occurrence of $P$ in $\beta$ ending at position $j$.

Obviously, $lcs(i-1, j)$ is equal to the length of the longest suffix of the string $\alpha[\,:\,i-1]$ pre-occurring in $\alpha_2$ at position $j$. Below we recall the definition of a suffix tree [1].

**Definition 4.** A *suffix tree* for a string $\beta$ of length $m$ is a directed tree with the following properties:

- The tree has $m$ leaves numbered from 1 to $m$;
- Each non-leaf vertex has at least two children;
- Every arc is labeled with a nonempty substring of $\beta$;
- The first symbols of labels of any two arcs going from one vertex are distinct;
- Reading the labels on arcs along the path from the root to a leaf $k$, one gets $\beta[k:]\$$.

Besides (explicit) vertices mentioned in the definition, we also consider "*implicit*" vertices, i.e., positions on an arc of the suffix tree corresponding to a single symbol written on the arc. To make the representation compact, arcs of a suffix tree are labeled with indices marking initial and final positions of substrings of $\beta$ rather than the substrings themselves. In this case only linear (in $\beta$) space is needed to store the suffix tree.

One says that a vertex of a suffix tree is labeled by a string $\gamma$ if $\gamma$ can be obtained by concatenating (in an appropriate order) all labels on the path from the root to this vertex.

There are many algorithms for constructing suffix trees. The most famous one is probably due to Ukkonen [3]. This algorithm has linear running time, and besides a suffix tree it also builds *suffix links*, which will be needed later.

**Definition 5.** A *suffix link* is a directed arc from an (explicit) vertex $u$ labeled by a string $x\gamma$ ($x \in \Sigma$, $\gamma$ is a string) to an (explicit) vertex labeled by $\gamma$.

We need the following lemma (for a proof, see[1]).

**Lemma 1.** *Existence of a vertex labeled by a string $x\gamma$ ($x \in \Sigma$, $\gamma$ is a string) implies existence of a vertex labeled by $\gamma$, where each of the two vertices can be either explicit or implicit. Moreover, if the former vertex is explicit, then the latter is explicit too.*

First we build a suffix tree $T_i$ for $\alpha_1[\,:i-1]$. With the help of $T_i$, we compute a pre-occurrence of a substring of $\alpha_1[\,:i-1]$ of the maximum length at every position $j$ of $\alpha_2$, $1 \le j \le n_2$.

Namely, we follow arcs starting from the root of $T_i$ to read $\alpha_2$. Assume that we cannot proceed. This means that we are either at an explicit vertex $v$ and there is no outgoing arc from this vertex whose label starts with a matching symbol, or we are at an implicit vertex $v$ and the next symbol on the arc does not match the next symbol of the string. Assume that $v$ is labeled by $\beta$. Then the next step is to jump to a vertex $u$ (possibly implicit) that is labeled by $\beta[2:]$. This is done as follows.

If $v$ is explicit, the suffix link from $v$ goes directly to $u$. If $v$ is implicit, we first go up to the explicit parent of $v$. Then we take the suffix link from this parent and find ourselves somewhere on the path from the root to $u$. Now we simply go down along the arcs choosing the arc whose label starts with the appropriate symbol at each (explicit) vertex. There is no need to compare all symbols of the string and labels, since no vertex has two arcs with coinciding first symbols. A detailed description of this technique (called the *skip/count method*) can be found in [1].

Once arrived at $u$, we proceed with traversal of $T_i$ starting from the first still-unmatched position of $\alpha_2$.

It follows from the description of the traversal procedure that the label $\alpha_1[p_j : t_j]$ of a vertex where we end after reading the position $j$ of $\alpha_2$ is the *longest substring* of $\alpha_1[\,: i-1]$ pre-occurring at position $j$ of $\alpha_2$. Recall that our aim is to compute the *longest suffix* of $\alpha_1[\,: i-1]$ pre-occurring at position $j$ of $\alpha_2$. Below we show how to do this knowing $\alpha_1[p_j : t_j]$.

The string $\alpha_1[p_j : t_j]$ is a suffix of $\alpha_1[\,: t_j]$. We want to compute the starting position of the longest suffix of $\alpha_1[\,: i-1]$ that is also a suffix of $\alpha_1[p_j : t_j]$. The following lemma is obviously implied by Definition 2.

**Lemma 2.** *The length of the longest suffix of $\alpha_1[\,: i-1]$ that is also a suffix of $\alpha_1[\,: t_j]$ is equal to $Z_{i-t_j}(\alpha_1[1 : i-1]^{-1})$, where $\alpha_1[1 : i-1]^{-1}$ is the reverse string of $\alpha_1[1 : i-1]$.*

Let $Z_{i-t_j}(\alpha_1[1 : i-1]^{-1}) = k_j$. Then $\alpha_1[i-k_j : i-1]$ pre-occurs at position $t_j$ of $\alpha_1$. Since suffixes of smaller lengths are suffixes of $\alpha_1[i-k_j : i-1]$, the strings $\alpha_1[i-k_j+1 : i-1]$, $\alpha_1[i-k_j+2 : i-1]$, …, $\alpha_1[i-1]$, and additionally $\alpha_1[p_j : t_j]$ pre-occur at position $t_j$ of $\alpha_1$. Define $r$ to be $\min(k_j, t_j - p_j + 1)$. Then the longest suffix of $\alpha_1[\,: i-1]$ that is also a suffix of $\alpha_2[\,: j]$ is $\alpha_1[i-r : i-1]$.

**Time and space analysis.** Here we estimate the time and space used by the LCS algorithm. First we estimate the time needed for traversing $T_i$.

**Definition 6.** The *depth* of a vertex is the number of explicit vertices on the path from the root of the tree to this vertex.

**Lemma 3.** *Let $u$ and $v$ be explicit vertices of a suffix tree, and let $(u, v)$ be the suffix link from $u$ to $v$. Then the vertex depth of $u$ exceeds the vertex depth of $v$ by at most 1.*

For a proof of the lemma, see[1]. The current vertex depth can be decreased by one when going up to the immediate explicit parent and also by one when taking the suffix link (Lemma 3). When we go down along some edge, the vertex depth increases by one. The maximum vertex depth is $O(n_1)$. The number of suffix links that we take during the traversal is $O(n_2)$. Therefore, the total number of operations is $O(2n_2 + n_1) = O(n_2)$. Recall that $O(n_1)$ time and space is needed to construct $T_i$ and also its suffix links (Ukkonen's algorithm [3]).

Note that $Z$-values for $\alpha_1[\,: i-1]^{-1}$ and $T_i$ can be computed in $O(n_1)$ time and space.

## 5. CONCLUSION

Descriptions of LCP and LCS algorithms were given in Sections 3 and 4. If a position of a mismatch in strings $\alpha_1$ and $\alpha_2$ is fixed, the algorithms compute the length of the longest common substring with one mismatch in these positions in time $O(|\alpha_2|)$. The maximum length for all possible mismatch positions is the length of the longest common substring with one mismatch. Hence, the devised algorithm computes the length of the longest common substring of two strings with one mismatch in time $O(n_1 n_2)$. The advantage of this algorithm is that it reads $\alpha_2$ sequentially from left to right.

The described technique can be applied to other cases as well. Assume that $\gamma_1$ differs from $\gamma_2$ by insertion of a symbol. Then, to extract the longest common substring, it is sufficient to compute

$lcp(i+1, j+1) + lcs(i-1, j)$ for all $i$ and $j$ and to find the maximum of these values. The same reasoning works for deletion of a symbol.

The authors are grateful to A.L. Semenov for fruitful discussions and valuable remarks.

## REFERENCES

1. Gusfield, D., *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*, Cambridge: Cambridge Univ. Press, 1997. Translated under the title *Stroki, derev'ya i posledovatel'nosti v algoritmakh: informatika i vychislitel'naya biologiya*, St. Petersburg: Nevskii Dialekt, 2003.

2. Aho, A.V., Hopcroft, J.E., and Ullman, J.D., *The Design and Analysis of Computer Algorithms*, Reading: Addison-Wesley, 1976. Translated under the title *Postroenie i analiz vychislitel'nykh algoritmov*, Moscow: Williams, 2003.

3. Ukkonen, E., On-Line Construction of Suffix Trees, *Algorithmica*, 1995, vol. 14, no. 3, pp. 249–260.